01Intro

Algorithmie

Algorithmes et principes de programmation

Dans le domaine informatique, il existe plein de métiers différents. Il y a des spécialistes réseaux, des gestionnaires de serveurs, des spécialistes de bases de données...
Bien sûr, comme le titre du cours le suggère, on parlera essentiellement de programmation dans ce cours. Certains diront donc que ce cours n'est pas utile aux futurs gestionnaires de bases de données ou aux fournisseurs systèmes et autres.
En réalité, la plupart des domaines informatiques utilisent des algorithmes et/ou la logique associée. Il est évident que tout les informaticiens ne sont pas programmeurs, mais celui qui n'arrive pas à comprendre un algorithme simple ne sera jamais un bon informaticien.
Ce cours est donc essentiel, quel que soit le métier informatique que vous voulez exercer.

Etant bien conscient que tout le monde ne veut pas devenir programmeur et qu'il y a des personnes qui débutent en informatique, le cours est vraiment à la portée de tous. Je m'engage à être toujours présent et à l'heure (sauf cas de force majeur), d'utiliser au mieux les moyens didactiques, d'être toujours motivé, de vous donner la possibilité de cours de remédiation deux fois par semaine...
MAIS, tout cela ne suffira pas sans un minimum de respect et de travail de votre part. Malheureusement, on a cours une fois par semaine et ce n'est pas évident de se rappeler de ce qu'on a fait la semaine précédente. Dès lors, il est primordial de relire le cours, les notes de la semaine précédente avant le cours.
Il est également logique que celui qui n'est pas présent à un cours, mette tout en œuvre pour rattraper son retard et ne retarde pas les autres.

Je ferai tout ce que je peux pour que ce cours soit fantastique et qu'il soit votre cours préféré, mais cela ne dépend pas que de moi...

02Variables

Les variables

En programmation, une variable est un emplacement mémoire auquel on a donné un nom et qui va nous permettre de stocker de l'information.

Il faut distinguer deux étapes à l'utilisation d'une variable: la déclaration et l'assignation.

La déclaration d'une variable est l'étape qui consiste à dire au système qu'on veut réserver un emplacement mémoire et lui attribuer un nom. Dans certains langages (Java, C#, par exemple), il faut préciser le type de données qu'on stockera dans cet emplacement. Cette étape de déclaration ne doit se faire qu'une fois par variable dans son scope.

L'assignation consiste à attribuer une valeur à un emplacement mémoire préalablement déclaré. Cette assignation se fait dans la plupart des langages avec le signe d'égalité. On peut assigner plusieurs fois des valeurs à la même variables, la donnée précédente étant remplacée par la nouvelle.

Les étapes de déclaration et d'assignation peuvent être réalisées en même temps, dans la même instruction.

Exemples de déclaration avec typage (C#):

- int i;
- bool isMarried;
- string firstname;

Exemples d'assignation (on admet que l'étape de déclaration a été effectuée):

- i = 3;
- isMarried = false;
- firstname = "David";

Exemples de déclaration et assignation immédiate (C#):

- int i = 3;
- bool isMarried = false;
- string firstname = "David";

Les noms donnés aux variables est important et doit suivre certaines règles. Celles-ci dépendent du langage de programmation. En PHP, par exemple, les noms de variables doivent débuter par le signe $.
Il y a aussi des mots réservés qu'on ne peut pas utiliser, mais ceux-ci sont dépendant du langage utilisé.
La plupart des langages n'acceptent pas non plus qu'on commence le nom par un chiffre ou un signe particulier (excepté souvent le signe underscore) et le nom ne peut pas comporter d'espace.
Au-delà de ces règles obligatoires, il y a aussi les conventions. Celles-ci sont aussi dépendantes du langage. Exemple: nom de variables commençant par minuscule pour Java, JavaScript... et commençant par une majuscule pour C# ou VB...
Comme les noms ne peuvent pas comporter d'espace, on utilise le camelCase ou le PascalCase. Ce principe veut que l'on place une lettre majuscule pour chaque mot lorsqu'on utilise un nom composé. Pour le camelCase, on commence par une minuscule et pour le PascalCase, par une majuscule.

Exemples de noms camelCase: beginDate, firstDayOfRetirement, lastYear...
Exemples de noms PascalCase: BeginDate, FirstDayOfRetirement, LastYear...

Certains langages privilégient le signe underscore pour séparer les mots, mais les professionnels préfèrent l'utiliser que dans certains cas précis.

Il est important de bien définir le nom de ses variables. Ceci augmente la lisibilité du code et permet de réduire les commentaires dans le code.

Comme on peut le constater dans les exemples ci-dessus, on doit indiquer le type de variable quand on déclare. Ce n'est pas toujours le cas dans tous les langages, mais ici oui. Celà va permettre au système de "réserver" une place qui fait "la bonne taille". Ci-dessous deux tableaux résumant les types de données en C# (venant de tutorialspoint et w3schools).

03Tests -IF

Les tests

Dans ce chapitre, lorsqu'on parle de "tests", on entend le principe d'algorithmie, de programmation et non le testing du logiciel ou partie de logiciel.

Le mot-clé des instructions de tests est dan beaucoup de langages "IF". En français, cela est simplement traduit par "SI". Comme dans le langage usuel courant, quand on commence une phrase par si, on sous-entend un test. On va comparer des choses ou voir **si** une condition est remplie avant d'exécuter une tâche. Le résultat d'un "si" est forcément toujours soit "oui", soit "non", autrement dit le résultat est toujours du type booléen (true ou false).

Exemple: Si (tu es gentil), je t'offre ce nouveau jeu.

Dans l'exemple, la réponse à "tu es gentil" ne peut être que vrai ou faux...un booléen.

En syntaxe C#, on écrira de cette façon: if(boolean expression){...}

Le mot-clé "if" sera donc toujours suivi de parenthèses dans lesquelles on aura un résultat booléen. Ensuite, on retrouve un bloc de code (entouré d'accolades) qui reprend les instructions à exécuter en cas de réponse positive du "si". Quand il n'y a qu'une instruction, les accolades sont facultatives, mais dans un souci de clarté, il est conseillé de toujours les mettre.

Exemples :

if(5 > 3){Console.WriteLine("cinq est plus grans que trois");}

if(a < b){Console.WriteLine("a est plus petit que b");}

La condition qui se trouve entre parenthèses peut être plus complexe, ce qui importe est le résultat: *true* ou *false*. Par extension, on peut aussi remplacer la condition par une variable booléenne.

Exemple:

```
bool isMarried = true;
if(isMarried){Console.WriteLine("Pas de chance");}
```

Bien que le "*if*" puisse exister "seul" comme dans les exemples précédent, il est souvent associé au mot-clé "else". Très logiquement, on retrouvera après le "*else*", un bloc de code entre accolades qui sera exécuté si la condition du "*if*" n'est pas remplie. Si on reprend l'exemple ci-dessus, on pourrait faire:

```
bool isMarried = true;
if(isMarried){
Console.WriteLine("Pas de chance");
}
else{
Console.WriteLine("Chouette, la vie de célibataire");
}
```

Parfois, on a besoin de plus de deux possibilités. On aimerait, par exemple, afficher un score en rouge s'il est en dessous de 50, en orange s'il est entre 50 et 60 et en vert au dessus de 60. Dans les cas semblables, on utilise un enchainement de *if...else if...else...* Attention à la syntaxe, en fonction du langage, le "*if*" et le "*else*" sont parfois fusionnés. Ici, en C# ce n'est pas le cas.

Exemple:

```
if(score<50){
Console.WriteLine("rouge");
}else if(score>=50 && score<60){
Console.WriteLine("orange");
else{
Console.WriteLine("vert");
}
```

Notez bien que on peut avoir autant de "*else if*" qu'on veut, on doit terminer par un "*else*". Vous aurez également remarqué le "&&" qui veut dire "et". Les deux conditions doivent être remplies. Le "*ou*" se note "||".

Souvent, une manière plus élégante que des if...else if... qui s'enchainent est le "switch case". Cette notation permet souvent plus de clarté. Voici un exemple:

```
int a = 3;

switch(a){

case 0: Console.WriteLine("0");break;

case 1: Console.WriteLine("1");break;
```

case 2: Console.WriteLine("2");break;

case 3: Console.WriteLine("3");break;

default: Console.WriteLine("other");break;

}

Dans cet exemple, la variable *a* va être évaluée. En fonction de la valeur décrite dans les différents cas, le code correspondant sera exécuté.

Enchainer les conditions

Ce que nous voulons dire par echainer les conditions, c'est exécuter quelque chose si plusieurs conditions sont remplies. Par exemple, je peux aller au cinéma si j'ai fini mes devoirs et si j'ai assez d'argent:

if(finiDevoirs et assezArgent){
je peux aller au cinéma
}

En code, ça donnerait quelque chose du genre:

```
if(finiDevoirs && assezArgent){
Console.WriteLine("Je peux aller au cinéma");
}
```

Dans cet exemple, les deux conditions doivent être remplies pour que le code s'exécute.

Parfois, on aimerait qu'une des deux conditions soient remplies. Dans ce cas, on parle de "ou" (or en anglais).
Le symbole est "||" dans ce cas. Voici un exemple:

```
if(majeur || autorisationParents){
Console.WriteLine("Je peux aller en Allemagne seul.");
}
```

Ce qui importe, c'est que le résultat de ce qui se trouve entre les parenthèse donne un boolean comme résultat, en français: le résultat doit être oui ou non.

Le Modulo

Le modulo est ce qu'on appelle généralement le reste de la division. Avec les années, on a tous appris à travailler avec des virgules. Ainsi, si on demande de diviser 5 par 2, on répond instinctivement 2 virgule 5. Mais souvenez-vous de vos premières années d'école. Au début, on disait que 5 diviser par 2 faisait 2 reste 1. Le reste de division de 5 par 2 est 1. Voici quelques exemples:

4/2 = 2 reste 0
7/2 = 3 reste 1
10/3 = 3 reste 1

23/3 = 7 reste 2
23/7 = 3 reste 2

En C#, la syntaxe est la suivante : 4%2, 7%2, 10%3...
Attention, ce qu'on abtient est bien **le reste** de la division et non le résultat !

Cette technique nous permet, par exemple de tester si un nombre est pair ou impair. En effet, tous les nombres qu'on **sait diviser par deux ont un reste de ... 0** !
En combinant cela avec notre "*if*", cela donne : *if((nombre%2) == 0){...}*

04Iterations - boucles

Les iterations - boucles

Après les variables et les tests (*if*), voici la troisième famille d'instructions en programmation. Il existe essentiellement deux grands types de boucles/itérations et chacune possède certaines variantes.

while(condition){...}

Le premier type de boucles est la "*while*" qui veut dire "tant que" en français. Il s'agit donc de répéter une ou des opérations tant qu'une condition est remplie. Exemple: Tant que l'utilisateur ne tape pas le bon code, on lui redemande le bon code.

La syntaxe de cette boucle peut différer selon le langage, mais reste très proche de la syntaxe utilisée ici en C#. En Java, par exemple, la syntaxe est identique.

Rien de tel qu'un petit exemple de code pour illustrer cette brève explication:

```
string pwd = "test123";
string userInput = "";
while(userInput != pwd){
Console.WriteLine("Introduisez votre code :");
userInput = Console.ReadLine();
}
```

Comme pour le *if*, entre les parenthèses, on retrouve la condition. Celà veut dire que le résultat de cette condition est *true* ou *false*, un booleen. Dans cet exemple, soit le mot de passe correspond (*true*), soit il ne correspond pas (*false*). Tant que le mot de passe ne correspond pas (d'où le signe *!=*), ce qui se trouve entre accolades est exécuté, répété.

do{...}while(condition);

Cette boucle est une variante de la précédente. Ici, la condition se retrouve à la fin du bloc de code, ce qui engendre la différence fondamentale avec la

boucle *while* "classique". En effet, il faut en déduire que le code de la boucle *do-while* sera exécuté toujours au moins un fois (ce n'est pas le cas pour la boucle while).

for(init;condition;increment/decrement){...}

Même si l'on peut toujours utiliser une boucle while, la boucle for sera préférée lorsqu'on doit faire des itérations un nombre limité de fois et/ou si un ou des compteur(s) est nécessaire.

Exemple: Afficher toutes les lettres d'un mot individuellement, chacune sur une ligne.

```
string mot = "test";
for(int i = 0; i < mot.Length;i++){
Console.WriteLine(mot[i]);
}
```

On peut initialiser plusieurs compteurs, avoir des conditions plus complexes (avec des && (and), || (or)...) et également incrémenter ou décrémenter par pas de deux ou trois...

```
for(int i = 0, int y = 10; i < 5; i++,y-=2){...}
```

Boucles foreach

Il s'agit d'une variante de la boucle for qui ne possède pas de compteur. La notation est plus simple qu'une boucle for classique, mais elle ne nous donne pas accès à un compteur. Elle est donc à privilégier lorsque nous n'avons pas besoin du compteur.

```
foreach(string x in tableauDeStrings){
 // traitement avec x, x équivaut à tableauDeStrings[i]
}
```

05Méthodes

Les méthodes

En fonction des langages, on appelle aussi les méthodes : fonctions, routines, opérations...

Les méthodes servent à regrouper des instructions sous un nom afin de pouvoir rappeler ces instructions plusieurs fois. Un autre but est de regrouper du code (des instructions) lié à une tache précise pour plus de clarté.

Dans les bonnes pratiques, on fait souvent référence au fait qu'un méthode ne doit faire qu'une chose et qu'elle doit comporter maximum 20 lignes. Bien sûr, ce sont des conseils généraux, pas toujours applicables. Cela dépend fort du contexte.

En C#, par convention, on utilise des noms de méthodes explicites en "Pascal-case". Ceci veut dire que la première lettre est en majuscules et que si le nom est composé de plusieurs mots, chaque nouveau mot commence par une majuscule (les espaces étant interdits, evidemment). Exemples: CalculerUnAge(), RendreLaMonnaie(), AfficherMenu()...

De par les exemples, vous remarquerez des parenthèses. Celles-ci sont obligatoires pour les méthodes, cela permet de les différencier des variables, par exemple.

Dans certains langages (PHP ou JavaScript, par exemple), on fait précéder le mot-clé "function" du nom de la méthode. En C# (ou Java), ce n'est pas le cas. Voyons cela au travers d'un exemple:
public static void AfficheMenu(){...}
Les deux premiers mot-clés, nous les verrons plus tard. Le troisième, "void" veut dire "rien" et correspond au type de retour. Il s'agit de ce que la méthode doit générer comme résultat. Dans cet exemple, la méthode doit afficher qqchose à la console, mais ne produit rien, ne retourne rien, d'où le "void".

Souvent, le but d'une méthode est générer un résultat qui doit être exploitable à l'extérieur de cette méthode. Dans ce cas, il faudra préciser le type de donnée générée à la place du "void". Ce type de donnée devra correspondre avec la donnée retournée et sera précédé du mot-clé "return". Exemple:
public static int GenererEntierAleatoire(){
int x = ...;
return x;
}

Si on fait l'analogie d'un algorithme avec une recette de cuisine, on peut considérer les analogies suivantes:

Faire une tarte aux pommes = algorithme

Prendre le livre de recettes = instruction

Prendre un bol = instruction

Verser du lait = instruction

...

On peut regrouper toutes les instructions qui servent à faire la pâte sous un nom: faire la pâte = méthode

On peut regrouper toutes les instructions qui servent à faire le dressage sous un nom: dressage = méthode

Comme écrit un peu plus haut, l'appel à une méthode peut générer un résultat. Exemple: faire de la pâte (méthode) donne comme résultat: de la pâte.

Faire appel à une méthode, c'est vouloir exécuter du code (des instructions). Parfois, vous n'avez pas besoin du résultat produit, comme par exmple: afficher un menu à l'écran.

Par contre, si vous demandez une addition ou une multiplication, il y a de fortes chances que le résultat vous intéresse. C'est alors qu'on a un "return".

Exemple d'une méthode qui donne le nombre pi:

```
public static double GivePi(){
return 3.14;
}
```

Il faut que le type de données retournée soit égal au type dans la signature de la méthode (ici double).

Autre exemple: nous voulons une méthode qui donne le nombre pi dans une phrase bien formulée:

```
public static string GivePiFrench(){
return "Le nombre pi est de " + 3.16;
}
```

A nouveau, le type de retour doit correspondre, ici c'est un string.

La plupart du temps, une méthode doit travailler sur des données venant "de l'extérieur". En effet, pour faire de la pâte, on doit verser du lait, ajouter des oeufs...

C'est ce qu'on appelle des paramètres. Voici un exemple illustratif:

```
public static Pate FairePate(bol, lait, oeufs, sucre, levure){
// je prends le bol y j'y verse le lait.
// j'ajoute le sucre et les oeufs
// je mélange
// j'ajoute la levure
return contenuDuBol;
}
```

En C#, les paramètres doivent être typés. Voici donc un exemple de méthode qui va multiplier deux nombres (x et y) et renvoyer le résultat:

```
public static int Multiplier(int x, int y){
return x*y; // x multiplié par y  donne un type int et correspond au int dans la d
éclaration
}
```

Autre exemple, une méthode qui concatène 3 strings et renvoie le résultat:

```
public static string Concatenate(string a, string b, string c){
return a + " "+b+" "+c;
}
```

06Arrays

Les tableaux - array

Les tableaux sont une structure de données permettant de regrouper plusieurs valeurs (de même type) dans une variable. Exemples: points obtenus à différents tests pour un étudiant, noms des étudiants d'une classe, les voyelles, les consonnes...

Il existe plusieurs façons d'initialiser un tableau (voir ci-dessous), mais ce qui est important à garder en mémoire, c'est que la **taille du tableau est figée à la création**. Ce n'est pas le cas dans tous les langages, mais bien en C# ou Java. Vous verrez que cela peut vite devenir un handicap.

Voyons la syntaxe de création et initialisation d'un tableau (array):

int[] points = new int[4];

Dans cet exemple, on déclare un tableau qui va contenir des entiers, on le nomme *points* et on l'initialise à une taille de 4. Notez les crochets, c'est la syntaxe untilisée pour dire tableau, array.
Comme signalé plus haut, la taille de ce tableau ne pourra plus varier. Bien sûr, on peut y placer 0, 1, 2 ou 3 entiers, mais la taille ne variera pas. C'est un peu comme si vous aviez une voiture qui peut contenir 5 personnes, mais rien ne vous oblige à toujours être à 5 personnes dans cette voiture.

Une fonctionnalité qui nous sera bien utile consiste à demander quelle taille fait le tableau. Cette fonctionnalité est une propriété du tableau et se récupère de la façon suivante:

int tailleDePoints = points.Length;

Notez ici qu'il n'y a pas de parenthèses après *Length*. S'il y avait eu des parenthèses, on aurait parlé de méthode et non de propriété.

Une deuxième façon de déclarer un tableau (en insérant deja des valeurs):

string[] noms = new string[]{"Eric","Albert","Marie","Sarah"};

Certains langages admettent également la notation raccourcie:

string[] noms ={"Eric","Albert","Marie","Sarah"};

Ici aussi, nous avons des tableaux avec une taille figée. La taille correspond au nombre d'éléments insérés dans le tableau. Nous pouvons donc écrire:

int tailleDeLaClasse = noms.Length; // ce qui donnera 4

Il nous reste à voir comment mettre/remplacer des éléments et comment retrouver des éléments dans un tableau. En considérant le tableau de noms ci-dessus, pour récupérer un nom, il faut indiquer l'index/indice. Par exemple, nous souhaitons

récupérer le nom *Albert*. Celui-ci se trouve à la position 1 (un tableau commence toujours à l'indice 0). Nous écrirons donc:

string nom1 = noms[1]; //la variable nom1 contient maintenant Albert

Pour mettre/remplacer un élément du tableau, il faut noter:

noms[2] = "Emeline"; // Emeline va se trouver à la position 2 à la place de Marie

Pour parcourir un tableau, on peut utiliser une boucle. Etant donné qu'on a besoin de l'indice, il est plus aisé d'utiliser une boucle *for*. Exemple:

```
for(int i = 0; i < noms.Length; i++){
Console.WriteLine(noms[i]);  // on affiche à la console, l'élément se trouvant à l'indice i
}
```

Notez que la condition de la boucle est strictement plus petit que. En effet, vu que la première position du tableau est 0, le dernier indice d'un tableau est toujours sa taille-1.

06Arrays-2

Les tableaux - array

Les tableaux sont une structure de données permettant de regrouper plusieurs valeurs (de même type) dans une variable. Exemples: points obtenus à différents tests pour un étudiant, noms des étudiants d'une classe, les voyelles, les consonnes...

Il existe plusieurs façons d'initialiser un tableau (voir ci-dessous), mais ce qui est important à garder en mémoire, c'est que la **taille du tableau est figée à la création**. Ce n'est pas le cas dans tous les langages, mais bien en C# ou Java. Vous verrez que cela peut vite devenir un handicap.

Voyons la syntaxe de création et initialisation d'un tableau (array):

int[] points = new int[4];

Dans cet exemple, on déclare un tableau qui va contenir des entiers, on le nomme *points* et on l'initialise à une taille de 4. Notez les crochets, c'est la syntaxe untilisée pour dire tableau, array.
Comme signalé plus haut, la taille de ce tableau ne pourra plus varier. Bien sûr, on peut y placer 0, 1, 2 ou 3 entiers, mais la taille ne variera pas. C'est un peu comme si vous aviez une voiture qui peut contenir 5 personnes, mais rien ne vous oblige à toujours être à 5 personnes dans cette voiture.

Une fonctionnalité qui nous sera bien utile consiste à demander quelle taille fait le tableau. Cette fonctionnalité est une propriété du tableau et se récupère de la façon suivante:

int tailleDePoints = points.Length;

Notez ici qu'il n'y a pas de parenthèses après *Length*. S'il y avait eu des parenthèses, on aurait parlé de méthode et non de propriété.

Une deuxième façon de déclarer un tableau (en insérant deja des valeurs):

string[] noms = new string[]{"Eric","Albert","Marie","Sarah"};

Certains langages admettent également la notation raccourcie:

string[] noms ={"Eric","Albert","Marie","Sarah"};

Ici aussi, nous avons des tableaux avec une taille figée. La taille correspond au nombre d'éléments insérés dans le tableau. Nous pouvons donc écrire:

int tailleDeLaClasse = noms.Length; // ce qui donnera 4

Il nous reste à voir comment mettre/remplacer des éléments et comment retrouver des éléments dans un tableau. En considérant le tableau de noms ci-dessus, pour récupérer un nom, il faut indiquer l'index/indice. Par exemple, nous souhaitons récupérer le nom *Albert*. Celui-ci se trouve à la position 1 (un tableau commence toujours à l'indice 0). Nous écrirons donc:

string nom1 = noms[1]; //la variable nom1 contient maintenant Albert

Pour mettre/remplacer un élément du tableau, il faut noter:

noms[2] = "Emeline"; // Emeline va se trouver à la position 2 à la place de Marie

Pour parcourir un tableau, on peut utiliser une boucle. Etant donné qu'on a besoin de l'indice, il est plus aisé d'utiliser une boucle *for*. Exemple:

```
for(int i = 0; i < noms.Length; i++){
Console.WriteLine(noms[i]);  // on affiche à la console, l'élément se trouvant à l'indice i
}
```

Notez que la condition de la boucle est strictement plus petit que. En effet, vu que la première position du tableau est 0, le dernier indice d'un tableau est toujours sa taille-1.

07Lists

Les listes != array

Les listes ne sont pas des tableaux. Leur but est sensiblement le même, mais l'utilisation est différente. Les tableaux sont une structure plus "basique" que les listes.

Nous savons que le plus gros point faible des tableaux (arrays) est qu'il ne sont pas redimensionnables (dans la plupart des langages). La taille est fixée à l'initialisation.

Pour ajouter au-dela de la taille du tableau, il faut recréer un nouveau tableau d'une taille supérieure et ensuite recopier le contenu et enfin ajouter la donnée voulue.

La liste va nous permettre de ne pas devoir gérer ce genre de choses. Elle va se redimensionner automatiquement. Nous devons juste nous habituer à une syntaxe un peu différente.

Bien qu'une liste pourrait contenir des objets de différents types (voitures, personnes, nombres...), il est conseillé (fortement!) d'utiliser des listes "typées". Cela veut dire qu'elle vont contenir le même type de données (ce qu'on retrouve entre <...> dans l'exemple ci-dessous).

Pour créer une liste:

List<string> maListe = new List<string>();

Vous remarquerez qu'ici, on utilise l'opérateur "new", pas forcément obligatoire pour la création d'un tableau.
Une fois la liste créée, on peut appeler des méthodes sur elle en utilisant son nom (de variable).
Par exemple:

maListe.RemoveAt(2);

Cette instruction va retirer l'objet se situant à l'indice 2 et la liste va se redimensionner automatiquement.
Pour ajouter à une liste:

maListe.Add("un nouveau string ou une varaible contenant un string");

Autre petite différence par rapports aux tableaux: souvenez-vous que pour connaitre la taille d'un tableau, on avait la propriété "Length". Pour les listes, cette propriété s'appelle "Count".

int sizeOfList = maListe.Count;

Pour accéder à un élément de la liste, on utilise les crochets comme pour le tableau:

string monString = maListe[2];

08Exceptions

Les exceptions

Ici, ce n'est qu'une petite introduction aux exceptions. Qu'est-ce qu'une exception ?

Il s'agit de quelque chose qui va interrompre le flux normal d'une application. Il faut faire la différence entre une erreur et une exception. Même si cette explication reste assez approximative, on peut résumer comme ceci:

Une erreur survient de façon inattendue et imprévisible, le programme est interrompu.

Une exception peut être prévisible, interromp le flux du programme, mais pas forcément de façon bloquante.

Des exemples d'erreurs pourraient être: mémoire saturée, coupure de courant, défaillance hardware...

Des exceptions...vous en avez déjà eu: transformer un string en int alors que ce n'est pas possible ou encore ajouter quelque chose à un tableau au-dela de sa taille...

En effet, prenons le cas de l'instruction que nous connaissons bien maintenant:

int age = Int32.Parse(monAge);

Que se passe-t-il si la variable monAge contient "W", par exemple ? Ici, il est clairement prévisible qu'une "erreur" (plus précisément une exception) peut survenir. Le système ne peut pas transformer un "W" en entier et il va donc interrompre le programme en nous signalant une exception.

Comme nous pouvons prévoir cette interruption de flux, nous pouvons "capturer" cette exception et traiter le cas où... En anglais, attraper est traduit par "catch", un mot-clé que nous allons utiliser.

Reprenons notre exemple de *int age = Int32.Parse(monAge);*

Nous ne savons pas forcément ce qui est contenu dans la varaible monAge. On s'attend à avoir qualque chose comme "32" ou "23"... mais rien n'empêche l'utilisateur à taper "W" si vous lui demandez son age.

Cette ligne de code peut donc potentiellement "faire planter" notre programme. Pour gérer ce problème et éviter un "plantage" de notre programme, un peut "entourer" le code pouvant générer une exception par un bloc "try" (avec accolades) et "capturer" l'exception si elle a lieu:

```
try{
int age = Int32.Parse(monAge);
}catch(Exception ex){
Console.WriteLine("Je gère...");
}
```

Que se passe-t-il dans le code ci-dessus ?

Et bien, si la variable contient une chaine de caractères pouvant être transformée en entier, rien de spécial. Si le système ne sait pas transformer la chaine de caractères, le bloc de code dans la partie "catch" sera exécutée.

09Lecture-ecriture fichier

Lecture & écriture de fichiers

Pour lire et écrire (fichiers, mémoire, réseau...), il faut ouvrir un flux (stream). C'est un peu comme un tunnel qu'on crée pour envoyer et recevoir des bytes. Quand on a fini, il faut refermer ce tunnel. La plupart des langages nous proposent des outils pour nous faciliter la tâche.
Ainsi, on distingue essentiellement deux types de flux: les flux de bytes (1 et 0) et les flux de caractères. Quand on lit ou écrit des caractères, ce sont aussi des 1 et des 0, mais il existe des flux qui transforment déjà pour nous ces 1 et 0 en caractères. Ici, nous ne verrons que les flux de lecture et écriture de caractères dans ou à partir de fichiers.

N'oubliez pas qu'une extension de fichier fait partie du nom de fichier et sert juste au système d'exploitation à savoir avec quel programme ouvrir ce genre de fichiers. Ainsi, vous pouvez inventer votre extension de fichier et si vous savez qu'il n'est composé que de caractères, vous pourrez les lire à partir d'un flux de caractères.

Lecture de fichier texte

Il existe plusieurs façons de lire un fichier, mais prenons la plus simple:

```
List<string> list = new List<string>();

using (StreamReader streamReader = new StreamReader(@"c:\file.txt"))

{

    string line;

    while ((line = streamReader.ReadLine()) != null)

    {

        list.Add(line);

    }

}
```

Ecriture de fichier texte

Ici aussi, il existe plusieurs façons de faire, surtout en C#. Le principe reste néanmoins toujours le même.

Ici, un exemple où grâce au "using" le flux est refermé automatiquement. Remarque: Si vous utilisez une solution sans le "using", il ne faut pas oublier de fermer le flux avec un "Close()".

Notez aussi le boolean pour dire qu'on veut ajouter une ligne au fichier (append=true), sinon il écrase ce qu'il y a déjà dedans.

```
using(StreamWriter writer = new StreamWriter(@"c:\temp\test.txt",true)){

writer.WriteLine("Une nouvelle ligne dans le fichier");

}
```

10Random

Objet Random

Derrière cet objet se cache un moyen de générer quelque chose d'aléatoire.

Générer un nombre complètement aléatoire est en réalité (à l'heure actuelle) impossible. C'est toujours un des défis mathématiques majeurs contemporains.

Mais ceci est une autre histoire...

En ce qui nous concerne, diverses techniques permettent de générer des nombres "considérés comme aléatoire". En général, elles se basent sur le temps, le moment.

En effet, si vous prenez la seconde écoulée, le centième de seconde écoulé... à un moment t, vous avez un nombre difficilement prévisible (ça dépend de l'échelle).

Assez de philosphie, passons aux choses concrètes.

En C#, une des façons de générer un nombre aléatoire est l'utilisation de l'objet Random. Celui-ci est très facile d'utilisation.

La chose principale à retenir est que votre objet doit être déclaré comme variable d'instance et non localement. Qu'est ce que ça veut dire ?

Comme tout objet, vous devez faire un "new" pour créer l'objet et maintenir cet objet (instance) dans une variable. Cette variable doit être déclarée en dehors des méthodes, directement dans la classe.

Concrètement, imaginez votre classe Program avec la méthode Main, voici la déclaration de l'objet Random:

```
public class Program{

static Random rand = new Random();

public static void Main(string[]args){

}

}
```

Comment interprêter cette nouvelle ligne de code ? Imaginez simplement que vous mettez dans la variable rand, un ami capable de vous aider en générant un nombre aléatoire.

Attention, cette ligne de code vous permet juste "d'accèder" à votre ami. par défaut, il ne fait rien.

Comment le solliciter pour avoir un nombre aléatoire ?

C'est très simple: votre ami (stocké dans la variable rand) comprend une commande (un ordre) qui est *Next()*.

Ainsi, si vous faites *rand.Next()* , il vous renvoie un int positif. Vous pouvez donc faire:

```
int x = rand.Next();
```

Mais le plus souvent, on veut générer un nombre aléatoire avec une certaine limite. Par exemple, je veux un nombre entier entre 0 et 10.

Dans ce cas, on peut le préciser:

```
int nombreAleatoireJusqueDix = rand.Next(11);
```

Dans l'exemple ci-dessus, vous constatez que j'ai mis 11. C'est parce que le nombre qu'on indique est exclu. En d'autres termes, c'est les nombres tant que < 11.

Par défaut, il commence à 0.

Si vous voulez un nombre plus spécifique, entre 3 et 6, par exemple, vous pouvez écrire:

```
int x = rand.Next(3,7);
```

Notez ici que le premier nombre dans les parenthèses est inclus.